

云原生训练营

模块十四： 基于 Kubernetes 和 Istio 的安全保证

孟凡杰

前 eBay 资深架构师



目录

1. 云原生语境下的安全保证
 - 容器运行时的安全保证
 - Kubernetes 的安全保证
2. Taint
3. NetworkPolicy
 - 依托于 Calico 的 NetworkPolicy
4. 零信任架构 (ZTA)
5. 基于 Istio 的安全保证
 - 认证
 - 鉴权

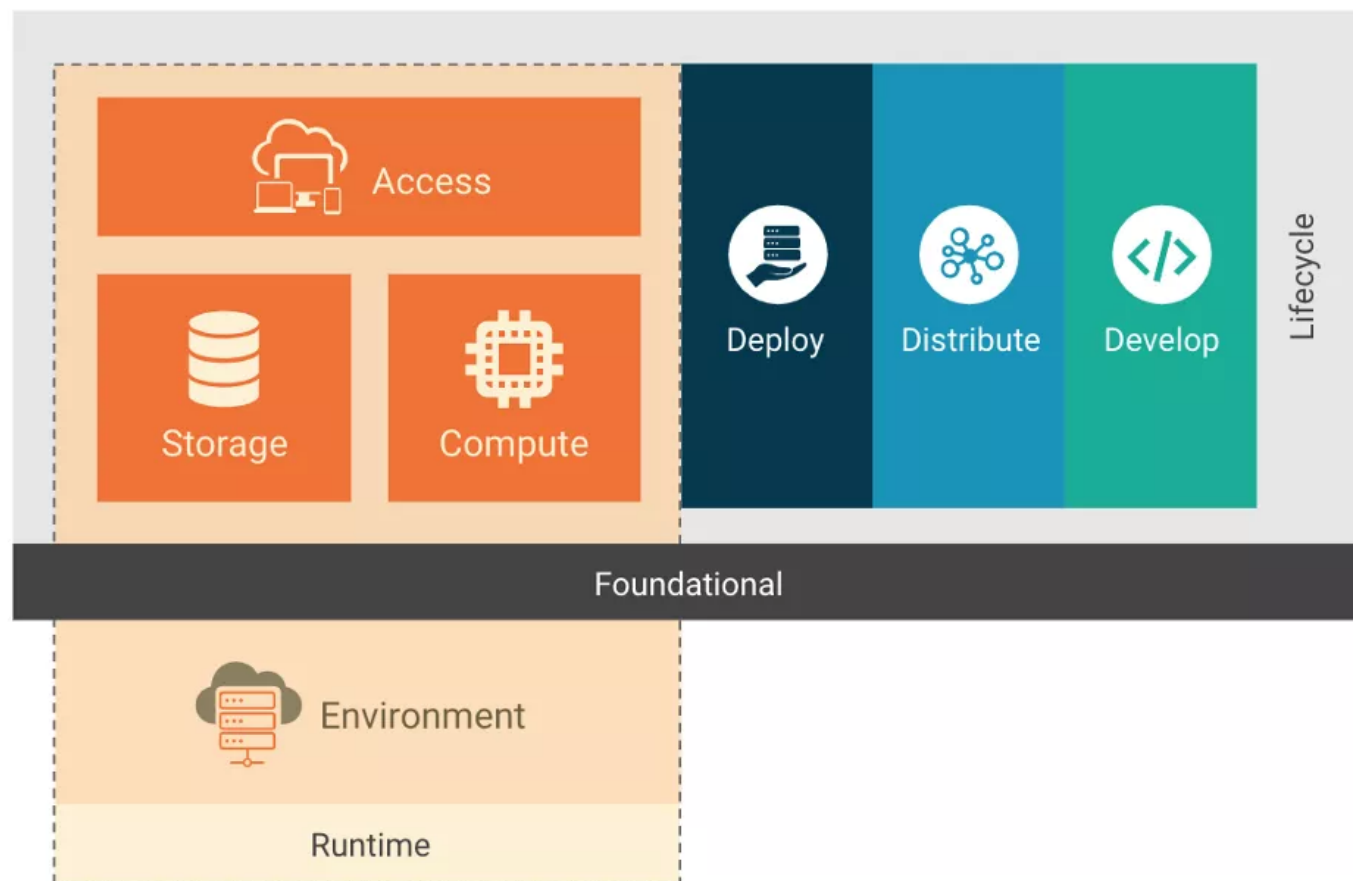
1. 云原生语境下的安全保证

云原生语境下的安全保证

- 安全保证是贯穿软件整个生命周期的重要部分。
- 安全与效率有时候是相违背的。
- 如何将二者统一起来，提升整体效率是关键。
- 这需要将我们将安全思想贯穿到软件开发运维的所有环节。

云原生层次模型

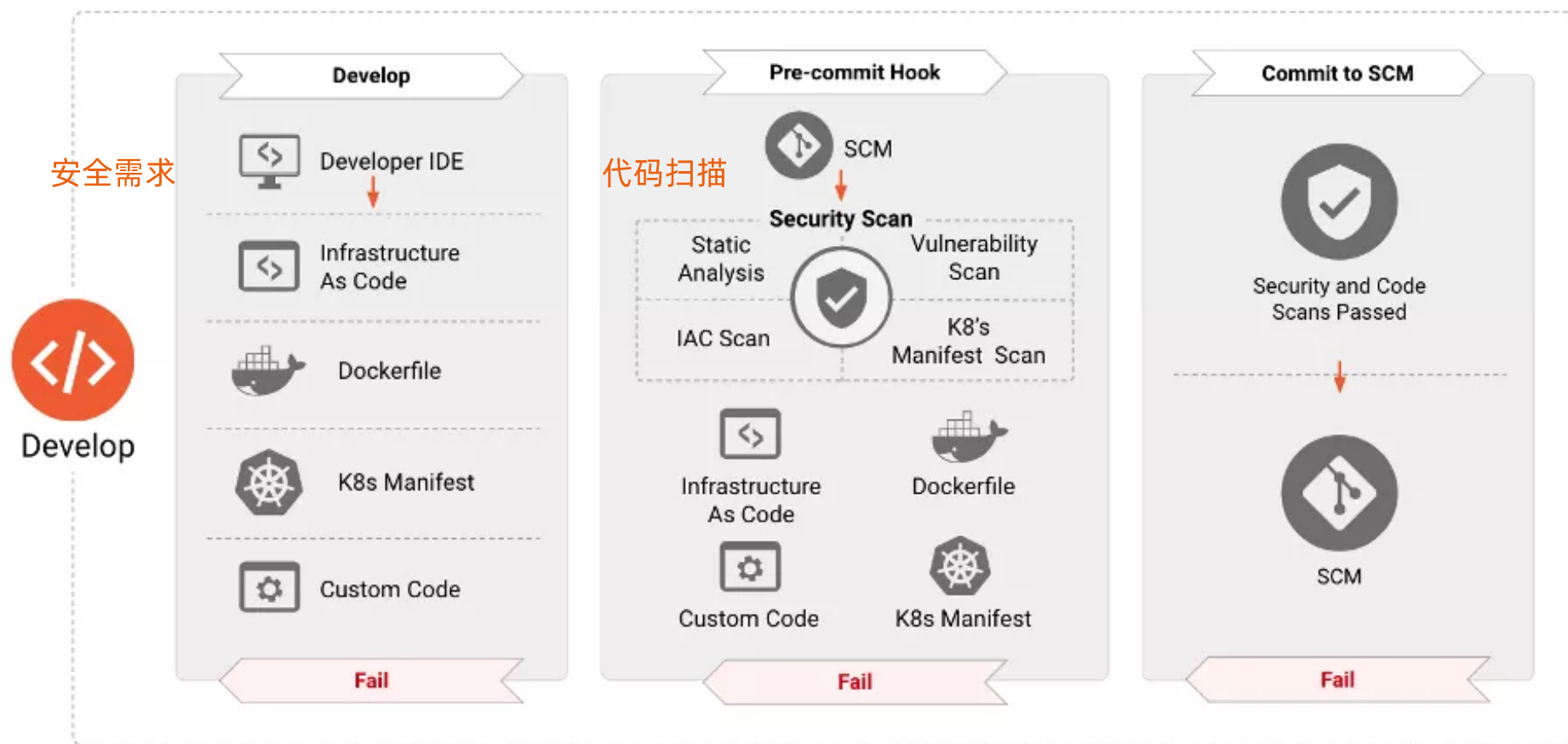
软件的生命周期：开发->分发->部署-> 运行



开发环节的安全保证

- SaaS 应用的 12-factor 设计原则的一些理念与云原生安全不谋而合。
- 传统的**安全三元素 CIA (Confidentiality, Integrity 和 Availability)**，在云原生安全中被充分应用，如对工作负载的完整性保护，与 I (Integrity) 完整性保护相对应。
 - 机密性 (Confidentiality) 指只有授权用户可以获取信息。
 - 完整性 (Integrity) 指信息在输入和传输的过程中，不被非法授权修改和破坏，保证数据的一致性。
 - 可用性 (Availability) 指保证合法用户对信息和资源的使用不会被不正当地拒绝。
- **基础设施即代码 (Infrastructure as Code, 简称 IaC)** 也与云原生的实践紧密相关。
- 这些方法和原则，都强调通过早期集成安全检测，以确保对过程的控制，使其按预期运行。
- 通过早期检测的预防性成本，降低后续的修复成本，提升了安全的价值。

开发



分发环节的安全保证

- 云原生应用生命周期中的分发阶段不仅需要包括验证工作负载本身的完整性的方法，还需要包括创建工作负载的过程和操作手段。
- 对于软件生产周期流水线中产生的工件（如容器镜像），需要进行持续的自动扫描和更新来确保安全，防止漏洞、恶意软件、不安全的编码和其他不安全的行为。
- 在完成这些检查后，更重要的是对产品进行加密签名，以确保产品的完整性及不可抵赖性。

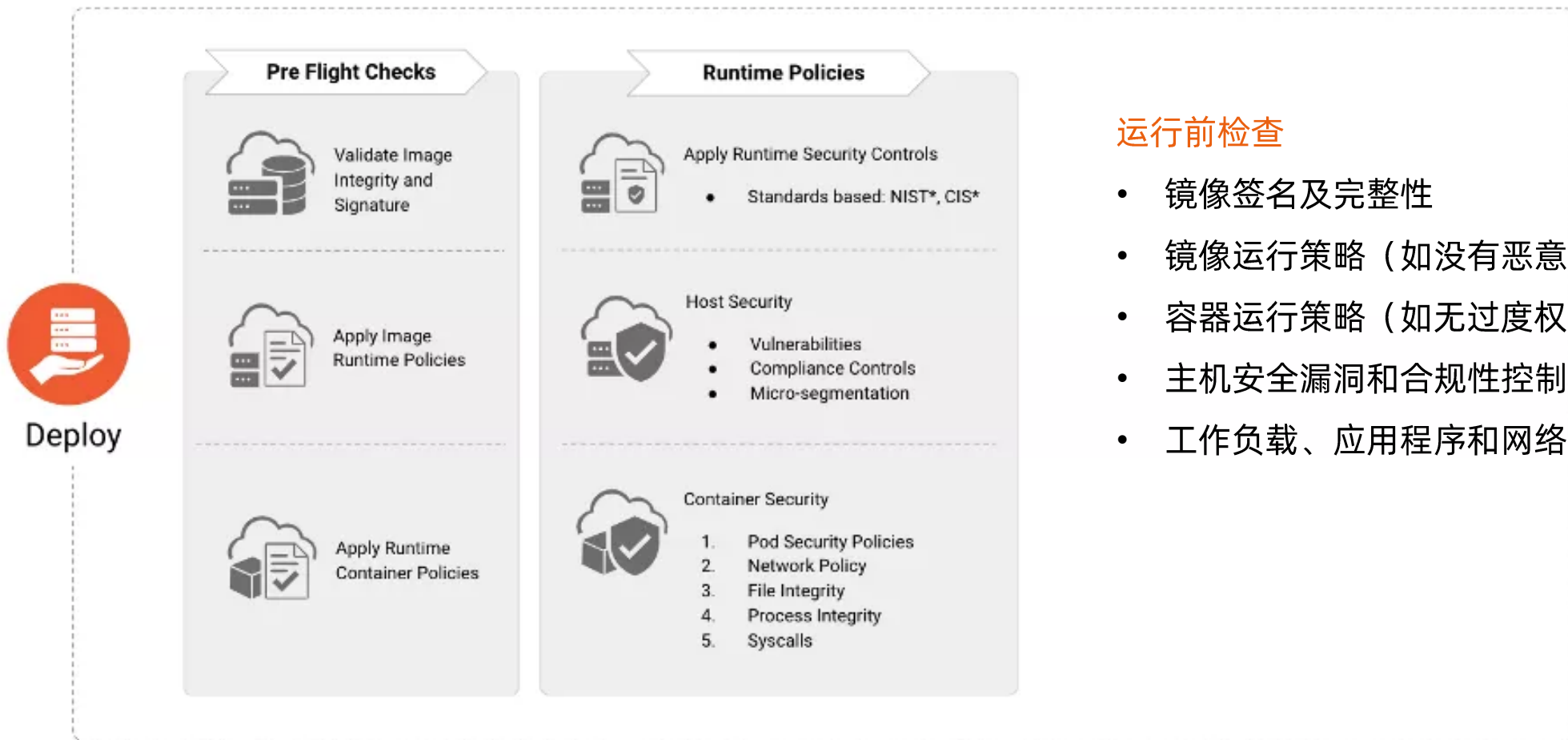
分发



部署环节的安全保证

- 在整个开发和集成发布阶段，应对候选工作负载的安全性进行实时和持续的验证，如，对签名的工件进行校验，确保容器镜像安全和运行时安全，并可验证主机的适用性。
- 安全工作负载的监控能力，应以可信的方式监控日志和可用指标，与工作负载一同部署来完善整体的安全性。

部署



运行前检查

- 镜像签名及完整性
- 镜像运行策略（如没有恶意软件或严重的漏洞）
- 容器运行策略（如无过度权限）
- 主机安全漏洞和合规性控制
- 工作负载、应用程序和网络安全策略

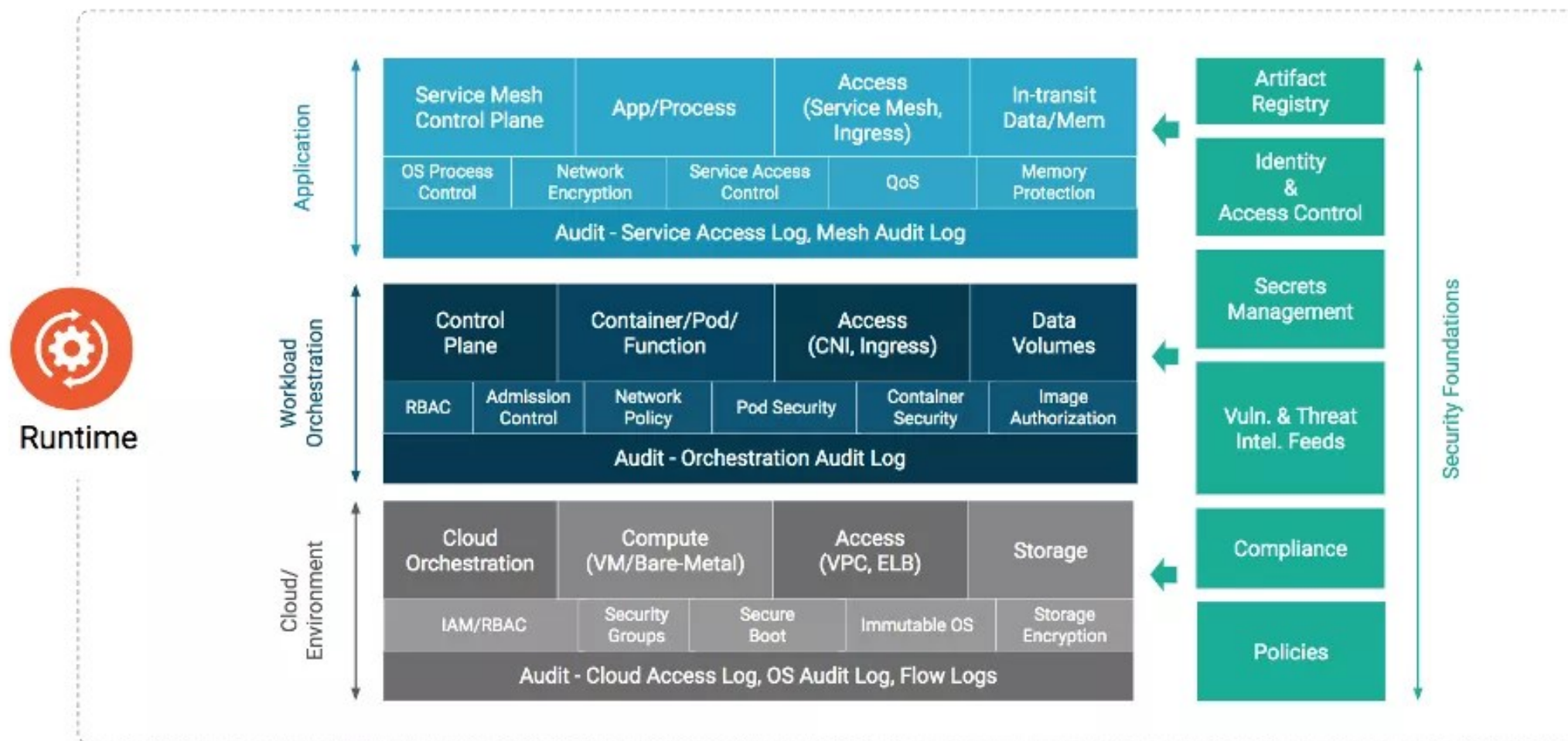
运行时环节的安全保证

应用程序通常由多个独立和单一职责的微服务组成，容器编排层使得这些微服务通过服务层抽象进行相互通信。

确保这种相互关联的组件架构安全的最佳实践，包括以下几点：

1. 只有经过批准的进程能在容器命名空间内运行；
2. 禁止并报告未经授权的资源访问；
3. 监控网络流量以检测恶意的活动；
4. 服务网格是另一种常见的服务层抽象，它为已经编排的服务提供了整合和补充功能，而不会改变工作负载软件本身（例如，API 流量的日志记录、传输加密、可观察性标记、认证和授权）。

运行环境



容器运行时的安全保障

以 Non-root 身份运行容器

在 Dockerfile 中通过 USER 命令切换成非 root 用户。

原因分析：

防止某些坏的镜像窃取主机的 root 权限并造成危害。

有些运行时容器内部的 root 用户与主机的 root 用户是同一个用户，如不进行用户切换很可能因为权限过大引发严重的问题，比如一个最简单的 case，主机上的重要文件夹被 mount 到容器内部，并被容器修改配置。

即使在容器内部也应该进行权限隔离，比如当我们希望构建不可变配置的容器镜像时，应该将运行容器的用户切换为非 root 用户，并且限制其读写权限和读写目录。

```
FROM ubuntu  
RUN user add patrick  
USER patrick
```

User Namespace 和 rootless container

- User Namespace:

- 依赖于 User namespace, 任何容器内部的用户都会映射成主机的非 root 用户。
- 但该功能未被默认 enable, 因其引入配置复杂性, 比如系统不知道主机用户和容器用户的映射关系, 在 mount 文件的时候无法设置适当的权限。

- Rootless container:

- rootless container 是指容器运行时以非 root 身份启动。
- 在该配置下, 即使容器被突破, 在主机层面获得的用户权限也是非 root 身份的用户, 这确保了安全。
- Docker 和其他运行时本身的后台 Daemon (如 Docker Daemon) 需要 root 身份运行, 然后其他用户的容器才能以 rootless 身份运行。
- 一些运行时, 比如 Podman, 无需 Daemon 进程, 因为可以完全不需要 root 身份。

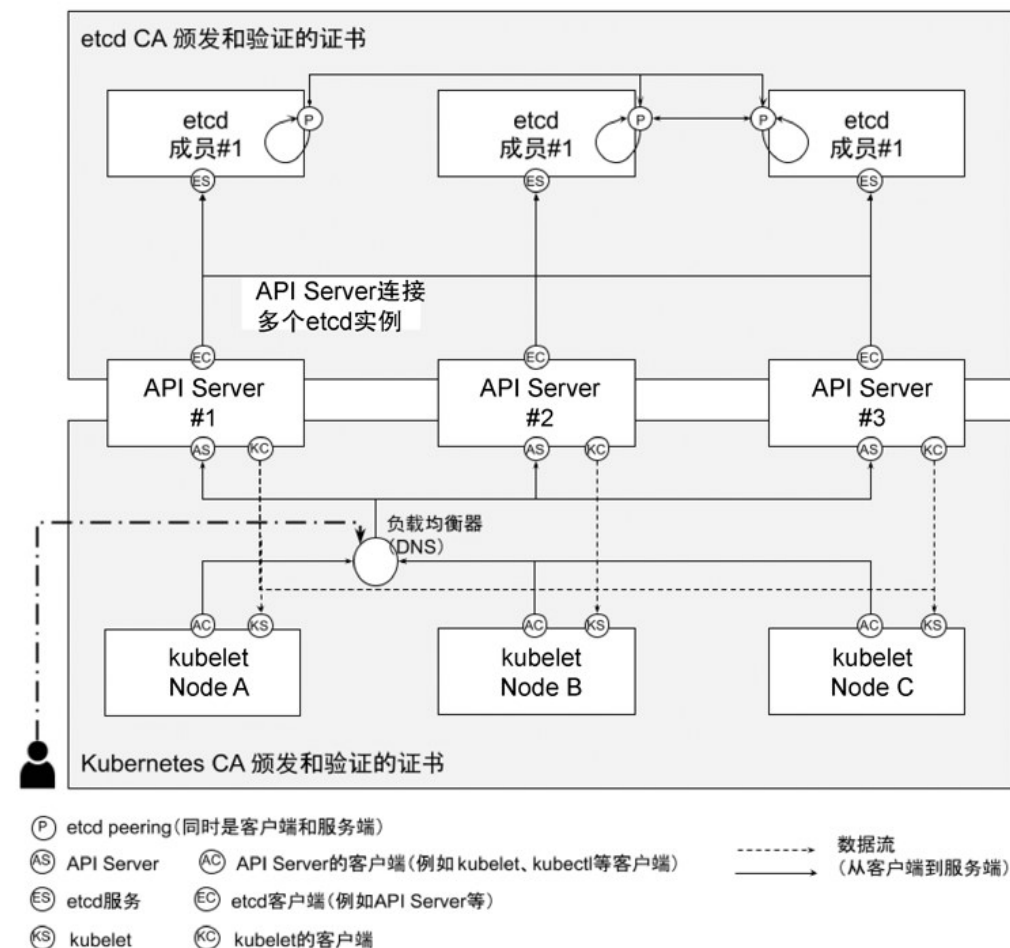
集群的安全性保证

- 保证容器与容器之间、容器与主机之间隔离，限制容器对其他容器和主机的消极影响。
- 保证组件、用户及容器应用程序都是最小权限，限制它们的权限范围。
- 保证集群的敏感数据的传输和存储安全。
- 常用手段
 - Pod安全上下文（Pod Security Context）
 - API Server的认证、授权、审计和准入控制
 - 数据的加密机制等

Kubernetes 的安全保证

集群的安全通信

Kubernetes 期望集群中所有的 API 通信在默认情况下都使用 TLS 加密，大多数安装方法也允许创建所需的证书并且分发到集群组件中。



控制面安全保证

- 认证

- 小型的单用户集群可能希望使用简单的证书或静态承载令牌方法。
- 更大的集群则可能希望整合现有的、OIDC、LDAP 等允许用户分组的服务器。
- 所有 API 客户端都必须经过身份验证，即使它是基础设施的一部分，比如节点、代理、调度程序和卷插件。这些客户端通常使用服务帐户或 X509 客户端证书，并在集群启动时自动创建或是作为集群安装的一部分进行设置。

- 授权

- 与身份验证一样，简单而广泛的角色可能适合于较小的集群，但是随着更多的用户与集群交互，可能需要将团队划分成有更多角色限制的单独的命名空间。

- 配额

- 资源配额限制了授予命名空间的资源的数量或容量。这通常用于限制命名空间可以分配的 CPU、内存或持久磁盘的数量，但也可以控制每个命名空间中有多少个 Pod、服务或卷的存在。

NodeRestriction

- 准入控制器限制了 kubelet 可以修改的 Node 和 Pod 对象，kubelet 只可修改自己的 Node API 对象，只能修改绑定到节点本身的 Pod 对象。
- NodeRestriction 准入插件可防止 kubelet 删除 Node API 对象。
- 防止 kubelet 添加/删除/更新带有 node-restriction.kubernetes.io/ 前缀的标签。
- 将来的版本可能会增加其他限制，以确保 kubelet 具有正常运行所需的最小权限集。

思考为什么？

- 降低获得 kubelet kubeconfig 的人能做成的破坏。

存储加密

```
apiVersion: API Server.config.k8s.io/v1
kind: EncryptionConfiguration
resources:
- resources:
- secrets
providers:
- identity: {}
- aesgcm:
  keys:
  - name: key1
    secret: c2VjcmV0IGlzIHNIY3VyZQ==
  - name: key2
    secret: dGhpcyBpcyBwYXNzd29yZA==
```

```
- aescbc:
  keys:
  - name: key1
    secret: c2VjcmV0IGlzIHNIY3VyZQ==
- secretbox:
  keys:
  - name: key1
    secret:
YWJjZGVmZ2hpamtsbW5vcHFyc3R1dnd4eXoxMjM0
NTY=
- kms:
  name : myKmsPlugin
  endpoint: unix:///tmp/socketfile.sock
  cachesize: 100
```

runAsUser

- SecurityContext
- PodSecurityPolicy
- MustRunAs
- MustRunAsNonRoot

Security Context

Pod 定义包含了一个安全上下文，用于描述允许它请求访问某个节点上的特定 Linux 用户（如 root）、获得特权或访问主机网络，以及允许它在主机节点上不受约束地运行的其它控件。

Pod 安全策略可以限制哪些用户或服务帐户可以提供危险的安全上下文设置。例如：Pod 的安全策略可以限制卷挂载，尤其是 hostpath，这些都是 Pod 应该控制的一些方面。

一般来说，大多数应用程序需要限制对主机资源的访问，他们可以在不能访问主机信息的情况下成功以根进程（UID 0）运行。但是，考虑到与 root 用户相关的特权，在编写应用程序容器时，你应该使用非 root 用户运行。

类似地，希望阻止客户端应用程序逃避其容器的管理员，应该使用限制性的 Pod 安全策略。

Kubernetes 提供了三种配置 Security Context 的方法：

- Container-level Security Context: 仅应用到指定的容器。
- Pod-level Security Context: 应用到 Pod 内所有容器以及 Volume。
- Pod Security Policies (PSP) : 应用到集群内部所有 Pod 以及 Volume。

Container-level Security Context

Container-level Security Context 仅应用到指定的容器上，并且不会影响 Volume。比如设置容器运行在特权模式：

```
apiVersion: v1
kind: Pod
metadata:
  name: hello-world
spec:
  containers:
    - name: hello-world-container
      # The container definition
      # ...
      securityContext:
        privileged: true
```

Pod-level Security Context

Pod-level Security Context 应用到 Pod 内所有容器，并且还会影响 Volume（包括 fsGroup 和 selinuxOptions）。

```
apiVersion: v1
kind: Pod
metadata:
  name: hello-world
spec:
  containers:
    # specification of the pod's containers
    # ...
  securityContext:
    fsGroup: 1234
    supplementalGroups: [5678]
    seLinuxOptions:
      level: "s0:c123,c456"
```

Pod Security Policies (PSP)

Pod Security Policies (PSP) 是集群级的 Pod 安全策略，自动为集群内的 Pod 和 Volume 设置 Security Context。

控制项	说明
privileged	运行特权容器
defaultAddCapabilities	可添加到容器的 Capabilities
requiredDropCapabilities	会从容器中删除的 Capabilities
volumes	控制容器可以使用哪些 volume
hostNetwork	host 网络
hostPorts	允许的 host 端口列表
hostPID	使用 host PID namespace
hostIPC	使用 host IPC namespace
seLinux	SELinux Context
runAsUser	user ID
supplementalGroups	允许的补充用户组
fsGroup	volume FSGroup
readOnlyRootFilesystem	只读根文件系统

PSP 示例

限制容器的 host 端口范围为 8000-8080:

```
apiVersion: extensions/v1beta1
kind: PodSecurityPolicy
metadata:
  name: permissive
spec:
  seLinux:
    rule: RunAsAny
  supplementalGroups:
    rule: RunAsAny
  runAsUser:
    rule: RunAsAny
  fsGroup:
    rule: RunAsAny
  hostPorts:
    - min: 8000
      max: 8080
  volumes:
    - '*'
```

2. Taint

为节点增加 taint

使用命令 `kubectl taint` 给节点增加一个 Taint:

```
kubectl taint nodes node1 key=value:NoSchedule
```

运行如下命令删除 Taint:

```
kubectl taint nodes node1 key:NoSchedule-
```

在 PodSpec 中为容器设定容忍标签:

tolerations:

- key: "key"

operator: "Equal"

value: "value"

effect: "NoSchedule"

tolerations:

- key: "key"

operator: "Exists"

effect: "NoSchedule"

Taint

可以以租户为粒度，为不同租户的节点增加 Taint，使得节点彼此隔离。

Taint 的作用是让租户独占节点，无对应 Toleration 的 Pod 无法被调度到 Taint 节点上，实现了应用部署的隔离。

3. NetworkPolicy

NetworkPolicy

如果你希望在 IP 地址或端口层面（OSI 第 3 层或第 4 层）控制网络流量，则你可以考虑为集群中特定应用使用 Kubernetes 网络策略（NetworkPolicy）。

Pod 可以通信的 Pod 是通过如下三个标识符的组合来辩识的：

- 其他被允许的 Pods；
- 被允许的名字空间；
- IP 组块。

网络策略通过网络插件来实现。要使用网络策略，你必须使用支持 NetworkPolicy 的网络解决方案。**创建一个 NetworkPolicy 资源对象而没有控制器来使它生效的话，是没有任何作用的。**

隔离和非隔离的 Pod

默认情况下，Pod 是非隔离的，它们接受任何来源的流量。

Pod 在被某 NetworkPolicy 选中时进入被隔离状态。

一旦名字空间中有 NetworkPolicy 选择了特定的 Pod，该 Pod 会拒绝该 NetworkPolicy 所不允许的连接。

网络策略不会冲突，它们是累积的。如果任何一个或多个策略选择了一个 Pod，则该 Pod 受限于这些策略的入站（Ingress）/出站（Egress）规则的并集。因此评估的顺序并不会影响策略的结果。

为了允许两个 Pods 之间的网络数据流，源端 Pod 上的出站（Egress）规则和目标端 Pod 上的入站（Ingress）规则都需要允许该流量。如果源端的出站（Egress）规则或目标端的入站（Ingress）规则拒绝该流量，则流量将被拒绝。

安全策略属性

spec: NetworkPolicy 规约中包含了在一个名字空间中定义特定网络策略所需的所有信息。

podSelector:

- 每个 NetworkPolicy 都包括一个 podSelector，它对该策略所适用的一组 Pod 进行选择。
- 空的 podSelector 选择名字空间下的所有 Pod。

policyTypes:

- 每个 NetworkPolicy 都包含一个 policyTypes 列表，其中包含 Ingress 或 Egress 或两者兼具。
- 如果 NetworkPolicy 未指定 policyTypes 则默认情况下始终设置 Ingress;
- 如果 NetworkPolicy 有任何出口规则的话则设置 Egress。

安全策略属性

Ingress:

- 每个 NetworkPolicy 可包含一个 Ingress 规则在白名单列表。
- 每个规则都允许同时匹配 from 和 ports 部分的流量。

Egress:

- 每个 NetworkPolicy 可包含一个 Egress 规则在白名单列表。
- 每个规则都允许匹配 to 和 port 部分的流量。

NetworkPolicy

apiVersion: networking.k8s.io/v1

kind: NetworkPolicy

metadata:

name: test-network-policy

namespace: default

spec:

podSelector:

matchLabels:

role: db

隔离 "default" 名字空间下 "role=db" 的 Pod

policyTypes:

- Ingress

- Egress

ingress:

- from:

- ipBlock:

cidr: 172.17.0.0/16

except:

- 172.17.1.0/24

- namespaceSelector:

matchLabels:

project: myproject

- podSelector:

matchLabels:

role: frontend

ports:

- protocol: TCP

port: 6379

"default" 名字空间下带有 "role=frontend" 标签的所有 Pod
带有 "project=myproject" 标签的所有名字空间中的 Pod
IP 地址范围为 172.17.0.0-172.17.0.255 和 172.17.2.0-172.17.255.255

egress:

- to:

允许从带有 "role=db" 标签的名字空间下的任何 Pod
到 CIDR 10.0.0.0/24 下 5978 TCP 端口

- ipBlock:

cidr: 10.0.0.0/24

ports:

- protocol: TCP

port: 5978

默认策略

默认拒绝所有入站流量

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-ingress
spec:
  podSelector: {}
  policyTypes:
  - Ingress
```

默认允许所有入站流量

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-all-ingress
spec:
  podSelector: {}
  ingress:
  - {}
  policyTypes:
  - Ingress
```

默认策略

默认允许所有出站流量

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-all-egress
spec:
  podSelector: {}
  egress:
  - {}
  policyTypes:
  - Egress
```

默认拒绝所有入口和所有出站流量

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-all
spec:
  podSelector: {}
  policyTypes:
  - Ingress
  - Egress
```

依托于 Calico 的 NetworkPolicy

NetworkPolicy

- NetworkPolicy 是命名空间级别资源。规则应用于与标签选择器匹配的 endpoint 的集合。

```
apiVersion: projectcalico.org/v3
```

```
kind: NetworkPolicy
```

```
metadata:
```

```
  name: allow-tcp-90
```

```
spec:
```

```
  selector: app == 'envoy' # 应用此策略的  
  endpoint
```

```
  types: # 应用策略的流量方向
```

```
    - Ingress
```

```
    - Egress
```

```
  ingress: # 入口的流量规则
```

```
    - action: Allow # 流量的行为
```

```
      protocol: ICMP # 流量的协议
```

```
      notProtocol: TCP # 匹配流量协议不为值的流量
```

```
      source: # 流量的来源 src 与 dst 的匹配关系为与，所有的都生效即生效
```

```
        nets: # 有效的来源 IP
```

```
        selector: # 标签选择器
```

```
        namespaceSelector: # 名称空间选择器
```

```
        ports: # 端口
```

```
          - 80 # 单独端口
```

```
          - 6040:6050 # 端口范围
```

```
      destination: # 流量的目标
```

```
  egress: # 出口的流量规则
```

```
    - action: Allow
```

```
  serviceAccountSelector: # 使用与此规则的 serviceAccount
```

GlobalNetworkPolicy

GlobalNetworkPolicy 与 NetworkPolicy 功能一样，是整个集群级别的资源。

GlobalNetworkPolicy 会在集群中所有 Namespace 生效，并且能限制主机（HostEndpoint）。

创建 NetworkPolicy 并查看结果

- 创建 networkpolicy

```
kubectl apply -f module14/networkpolicy.yaml
```

- 查看创建结果

```
kubectl get po -n ns-calico-01
```

- 登陆toolbox 容器

```
kubectl exec -it toolbox-68f79dd5f8-4664n bash
```

- 测试网络

```
ping 192.168.119.84
PING 192.168.119.84 (192.168.119.84)
56(84) bytes of data.
^C
--- 192.168.119.84 ping statistics ---
```

理解 Calico 的防火墙规则

```
:cali-FORWARD - [0:0]
```

```
:cali-INPUT - [0:0]
```

```
:cali-OUTPUT - [0:0]
```

```
-A INPUT -m comment --comment "cali:Cz_u1IQiXIMmKD4c" -j cali-INPUT
```

```
-A cali-INPUT -p udp -m comment --comment "cali:w7ud0UgQSEi_zKuQ" -m comment --comment "Allow VXLAN packets from whitelisted hosts" -m multiport --dports 4789 -m set --match-set cali40all-vxlan-net src -m addrtype --dst-type LOCAL -j ACCEPT
```

```
-A cali-INPUT -p udp -m comment --comment "cali:4cgmbdWsLmozYhjH" -m comment --comment "Drop VXLAN packets from non-whitelisted hosts" -m multiport --dports 4789 -m addrtype --dst-type LOCAL -j DROP
```

```
-A cali-INPUT -i cali+ -m comment --comment "cali:t45BUBhpu3Wsmi1_" -g cali-wl-to-host
```

```
-A cali-INPUT -m comment --comment "cali:NOmsycyknYZaGOFF" -m mark --mark 0x10000/0x10000 -j ACCEPT
```

```
-A cali-INPUT -m comment --comment "cali:Or0B7eoenKO2p8Bf" -j MARK --set-xmark 0x0/0xf0000
```

```
-A cali-INPUT -m comment --comment "cali:AmIfvPGG2lYUK6mj" -j cali-from-host-endpoint
```

```
-A cali-INPUT -m comment --comment "cali:79fWWn1SpufdO7SE" -m comment --comment "Host endpoint policy accepted packet." -m mark --mark 0x10000/0x10000 -j ACCEPT
```

```
-A cali-wl-to-host -m comment --comment "cali:Ee9Sbo10IpVujdlY" -j cali-from-wl-dispatch
```

```
-A cali-wl-to-host -m comment --comment "cali:nSZbcOoG1xPONxb8" -m comment --comment "Configured DefaultEndpointToHostAction" -j ACCEPT
```

```
-A cali-to-wl-dispatch -o cali2+ -m comment --comment "cali:ZJts1ClWTzhqZEeH" -g cali-to-wl-dispatch-2
```

```
-A cali-to-wl-dispatch-2 -o cali23a582ef038 -m comment --comment "cali:3OpBnQAsm2jeqKzt" -g cali-tw-cali23a582ef038
```

创建 NetworkPolicy 并查看结果

```
-A cali-tw-cali23a582ef038 -m comment --comment "cali:BCljDQubOGHklAW3" -m conntrack --ctstate RELATED,ESTABLISHED -j ACCEPT
-A cali-tw-cali23a582ef038 -m comment --comment "cali:yw1bAkXtcDncZYnn" -m conntrack --ctstate INVALID -j DROP
-A cali-tw-cali23a582ef038 -m comment --comment "cali:Q8lAtOCV68p7YEgj" -j MARK --set-xmark 0x0/0x10000
-A cali-tw-cali23a582ef038 -m comment --comment "cali:MIlFD0RDk3GLGBw_" -m comment --comment "Start of policies" -j MARK --set-xmark 0x0/0x20000
-A cali-tw-cali23a582ef038 -m comment --comment "cali:nuuVD6tedS2uGLLB" -m mark --mark 0x0/0x20000 -j cali-pi-_6MPzWU00Ko_GzNULnUn
-A cali-tw-cali23a582ef038 -m comment --comment "cali:1kxCb6jlwvgj1o-0" -m comment --comment "Return if policy accepted" -m mark --mark 0x10000/0x10000 -j RETURN
-A cali-tw-cali23a582ef038 -m comment --comment "cali:CjY7cfpY1dabGkAX" -m comment --comment "Drop if no policies passed packet" -m mark --mark 0x0/0x20000 -j DROP
-A cali-tw-cali23a582ef038 -m comment --comment "cali:w-x0nOtZNjQxVI9R" -j cali-pri-kns.ns-calico-01
-A cali-tw-cali23a582ef038 -m comment --comment "cali:coaStO4Ck_4QXbUO" -m comment --comment "Return if profile accepted" -m mark --mark 0x10000/0x10000 -j RETURN
-A cali-tw-cali23a582ef038 -m comment --comment "cali:df0Q_dYlf9LQgsi8" -j cali-pri-_sZa7qKPN6oyjdl6bLA
-A cali-tw-cali23a582ef038 -m comment --comment "cali:qFaig6sp7efC5L7x" -m comment --comment "Return if profile accepted" -m mark --mark 0x10000/0x10000 -j RETURN
-A cali-tw-cali23a582ef038 -m comment --comment "cali:R5lu-jBmqpEwUzOp" -m comment --comment "Drop if no profiles matched" -j DROP
```

创建规则允许 ICMP

- 创建规则允许 ICMP 协议

```
kubectl apply -f module14/allow-icmp-incluster.yaml
```

- 查看结果

```
nsenter -t 5989 -n ping 192.168.119.84
PING 192.168.119.84 (192.168.119.84) 56(84) bytes
of data.
64 bytes from 192.168.119.84: icmp_seq=1 ttl=63
time=0.061 ms
64 bytes from 192.168.119.84: icmp_seq=2 ttl=63
time=0.071 ms
```

查看规则变化

```
-A cali-tw-cali23a582ef038 -m comment --comment "cali:BCIjDQUBOGHklAW3" -m conntrack --ctstate RELATED,ESTABLISHED -j ACCEPT
-A cali-tw-cali23a582ef038 -m comment --comment "cali:yw1bAkXtcDncZYnn" -m conntrack --ctstate INVALID -j DROP
-A cali-tw-cali23a582ef038 -m comment --comment "cali:Q8lAtOCV68p7YEgj" -j MARK --set-xmark 0x0/0x10000
-A cali-tw-cali23a582ef038 -m comment --comment "cali:MLIFD0RDk3GLGBw_" -m comment --comment "Start of policies" -j MARK --set-xmark 0x0/0x20000
-A cali-tw-cali23a582ef038 -m comment --comment "cali:nuuVD6tedS2uGLLB" -m mark --mark 0x0/0x20000 -j cali-pi-_6MPzWUO0Ko_GzNULnUn
-A cali-tw-cali23a582ef038 -m comment --comment "cali:1kCb6jIwvgJ1o-0" -m comment --comment "Return if policy accepted" -m mark --mark 0x10000/0x10000 -j RETURN
-A cali-tw-cali23a582ef038 -m comment --comment "cali:8x_kaSR_adjgPMpR" -m mark --mark 0x0/0x20000 -j cali-pi-_1GuvCYJ2yW4bHzL8wH-
-A cali-tw-cali23a582ef038 -m comment --comment "cali:Rgn8KAKSp5qogm_L" -m comment --comment "Return if policy accepted" -m mark --mark 0x10000/0x10000 -j RETURN
-A cali-tw-cali23a582ef038 -m comment --comment "cali:MPNeMXPoJG9o70Z" -m comment --comment "Drop if no policies passed packet" -m mark --mark 0x0/0x20000 -j DROP
-A cali-tw-cali23a582ef038 -m comment --comment "cali:NQY_ueeL5deBM7Gr" -j cali-pri-kns.ns-calico-01
-A cali-tw-cali23a582ef038 -m comment --comment "cali:uXJa3c-DvyNMk4k2" -m comment --comment "Return if profile accepted" -m mark --mark 0x10000/0x10000 -j RETURN
-A cali-tw-cali23a582ef038 -m comment --comment "cali:Fvd66tSiXb2mjtmi" -j cali-pri-_sZa7qKPN6oyJdl6bLA
-A cali-tw-cali23a582ef038 -m comment --comment "cali:BFE1X5N6HK2ULI3F" -m comment --comment "Return if profile accepted" -m mark --mark 0x10000/0x10000 -j RETURN
-A cali-tw-cali23a582ef038 -m comment --comment "cali:1ROdF_sc6w3fUpLs" -m comment --comment "Drop if no profiles matched" -j DROP
-A cali-pi-_1GuvCYJ2yW4bHzL8wH- -p icmp -m comment --comment "cali:--CtA5sGB7G86H8e" -m set --match-set cali40s:5y5I3VdRZfDU01O--xXAPx2 src -m icmp --icmp-type 8 -j MARK --set-xmark 0x10000/0x10000
-A cali-pi-_1GuvCYJ2yW4bHzL8wH- -m comment --comment "cali:1Sl0swLbnXZNbBt9" -m mark --mark 0x10000/0x10000 -j RETURN
```

4. 零信任架构（ZTA）

传统安全模型

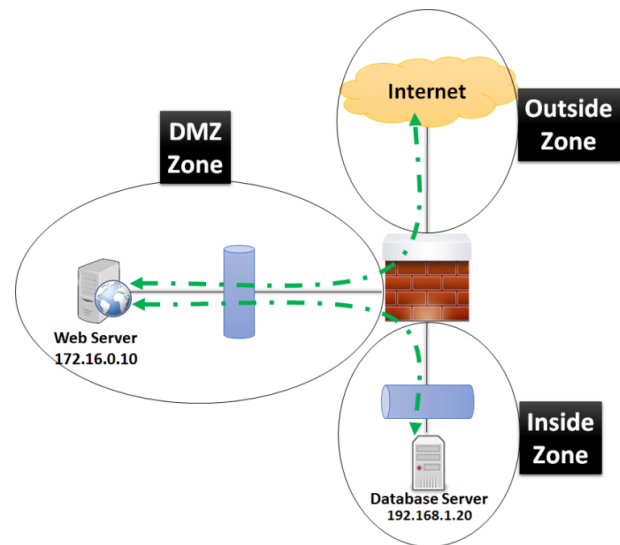
DMZ 模式

传统的网络安全架构理念是基于边界的安全架构，企业构建网络安全体系时，首先寻找安全边界，把网络划分为外网、内网、DMZ（DeMilitarized Zone）区等不同的区域，然后在边界上部署防火墙、入侵检测、WAF 等产品。

这种网络安全架构假设或默认了内网比外网更安全，在某种程度上预设了对内网中的人、设备和系统的信任，忽视加强内网安全措施。

不法分子一旦突破企业的边界安全防护进入内网，会像进入无人之境，将带来严重的后果。

传统的认证，即信任、边界防护、静态访问控制、以网络为中心。



零信任架构（Zero Trust Architecture, ZTA）

随着云计算、大数据、物联网、移动办公等新技术与业务的深度融合，网络安全边界也逐渐变得更加模糊，传统边界安全防护理念面临巨大挑战。

零信任核心原则：从不信任，始终验证

动态安全架构：

- 以身份为中心
- 以识别、持续认证、动态访问控制、授权、审计以及监测为链条
- 以最小化实时授权为核心
- 以多维信任算法为基础
- 认证达末端



零信任架构（ZTA）

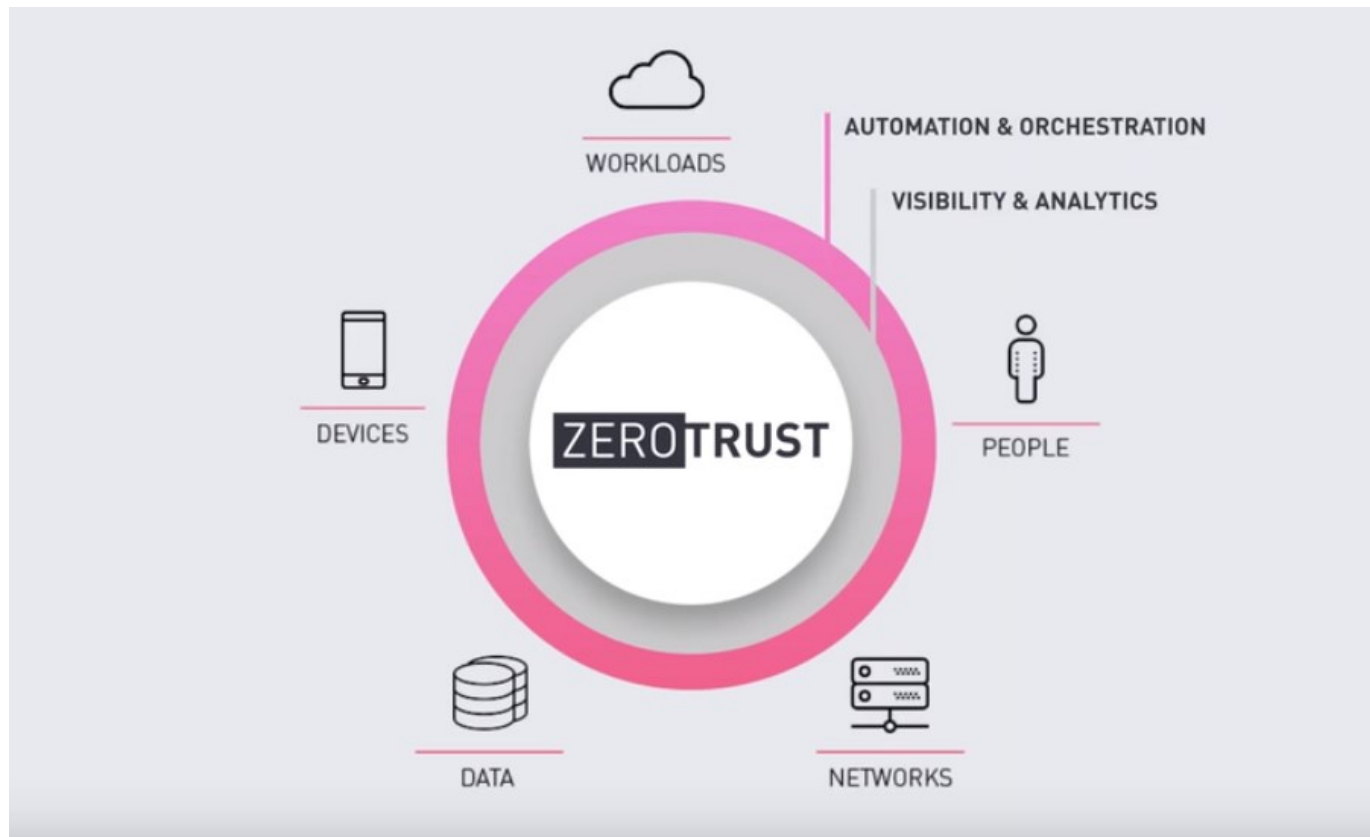
与边界模型的“信任但验证”不同，零信任的核心原则是“从不信任、始终验证”。

根据 Evan Gilman《Zero Trust Networks》书中所述，零信任网络建立在五个假设前提之下：

- 应该始终假设网络充满威胁；
- 外部和内部威胁每时每刻都充斥着网络；
- 不能仅仅依靠网络位置来确认信任关系；
- 所有设备、用户、网络流量都应该被认证和授权；
- 访问控制策略应该动态地基于尽量多的数据源进行计算和评估。

ZTA 安全模型

- 零信任数据
- 零信任人员
- 零信任网络
- 零信任工作负载
- 零信任设备
- 可视化和分析
- 自动化和编排



零信任架构的三大技术 “SIM”

零信任架构的三大技术 “SIM” ， 即

- 软件定义边界 (SDP, Software Defined Perimeter)
- 身份识别与访问管理 (IAM, Identity and Access Management)
- 微隔离 (MSG, Micro Segmentation)

软件定义边界（SDP）

- SDP 旨在使应用程序所有者能在需要时部署安全边界，以便将服务与不安全的网络隔离开。
- SDP 将物理设备替换为在应用程序所有者控制下运行的逻辑组件，仅在设备验证和身份验证后才允许访问企业应用基础架构。
- 基于 SDP 的系统通常会实施控制层与数据层的分离：
 - 控制流阶段，用户及其设备进行预认证来获取丰富的属性凭据作为身份主体，以此结合基于属性的预授权策略，映射得到仅供目标访问的特定设备和服务；
 - 数据传输阶段直接建立相应安全连接并传输数据。

身份识别与访问管理（IAM）

- 零信任强调基于身份的信任链条，即该身份在可信终端，该身份拥有权限才可对资源进行请求。
- 传统的 IAM 系统可以协助解决身份唯一标识、身份属性、身份全生命周期管理的功能问题。
- 通过 IAM 将身份信息（身份吊销离职、身份过期、身份异常等）传递给零信任系统后，零信任系统可以通过 IAM 系统的身份信息来分配相应权限。
- 通过 IAM 系统对身份的唯一标识，可有利于零信任系统确认用户可信，通过唯一标识对用户身份建立起终端、资源的信任关系，并在发现风险时实施针对关键用户相关的访问连接进行阻断等控制。

微隔离（MSG）

- 微隔离本质上是一种网络安全隔离技术
 - 能够在逻辑上将数据中心划分为不同的安全段，一直到各个工作负载级别；
 - 为每个独立的安全段定义访问控制策略。
- 它主要聚焦在云平台东西向流量的隔离
 - 一是区别传统物理防火墙的隔离作用；
 - 二是更加贴近云计算环境中的真实需求。
- 微隔离将网络边界安全理念发挥到极致，将网络边界分割到尽可能的小，能够很好的缓解传统边界安全理念下的边界过度信任带来的安全风险。

5. 基于 Istio 的安全保证

微服务架构下的安全挑战

为了抵御中间人攻击，需要流量加密。

为了提供灵活的服务访问控制，需要双向 TLS 和细粒度的访问策略。

要确定谁在什么时候做了什么，需要审计工具。

Istio 的安全保证

Istio 安全功能提供：

身份识别；

灵活策略；

透明的 TLS 加密；

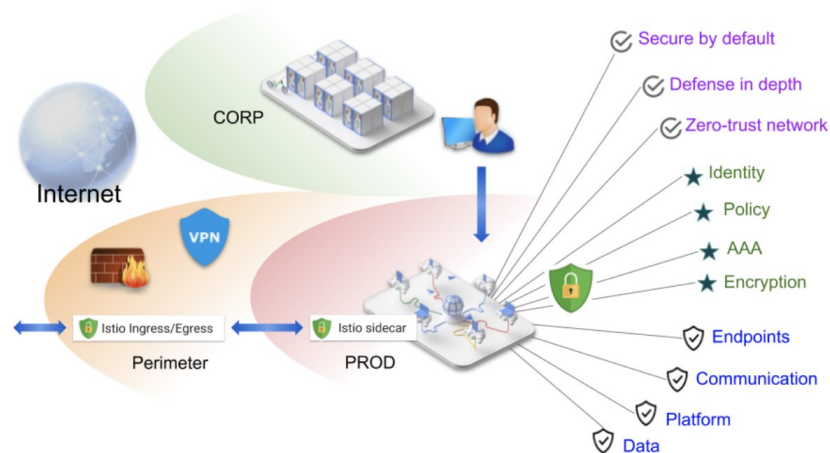
认证，授权和审计（AAA）工具来保护你的服务和数据。

Istio 安全的目标是：

默认安全：应用程序代码和基础设施无需更改。

深度防御：与现有安全系统集成以提供多层防御。

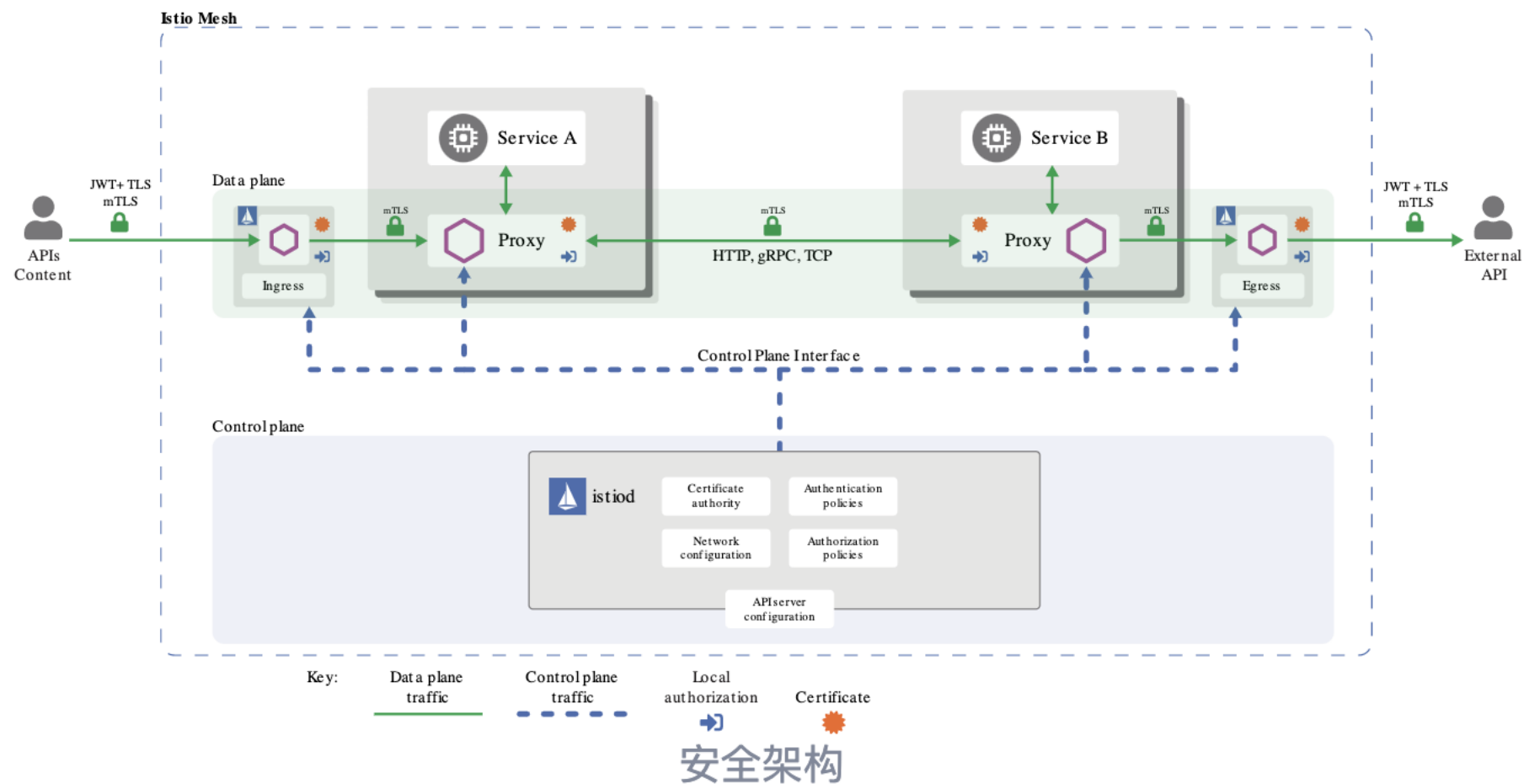
零信任网络：在不受信任的网络上构建安全解决方案。



高层架构

- 用于密钥和证书管理的证书颁发机构（CA）。
- 配置 API 服务器分发给代理：
 - 认证策略
 - 授权策略
- 安全命名信息
- Sidecar 和边缘代理作为 Policy Enforcement Points(PEPs) 以保护客户端和服务端之间的通信安全。
- 一组 Envoy 代理扩展，用于管理遥测和审计。

Istio 安全架构



Istio 身份

身份是任何安全基础架构的基本概念。

在工作负载间通信开始时，双方必须交换包含身份信息的凭证以进行双向验证。

在客户端，根据安全命名信息检查服务器的标识，以查看它是否是该服务的授权运行程序。

在服务器端，服务器可以根据授权策略确定客户端可以访问哪些信息，审计谁在什么时间访问了什么，根据他们使用的工作负载向客户收费，并拒绝任何未能支付账单的客户访问工作负载。

Istio 身份模型使用 service identity（服务身份）来确定一个请求源端的身份。

Istio 身份

Kubernetes: Kubernetes 服务帐户

GKE/GCE: 可以使用 GCP 服务帐户

GCP: GCP 服务帐户

AWS: AWS IAM 用户/角色 帐户

On-premises (非 Kubernetes) : 用户帐户、自定义服务帐户、服务名称、Istio 服务帐户或 GCP 服务帐户。

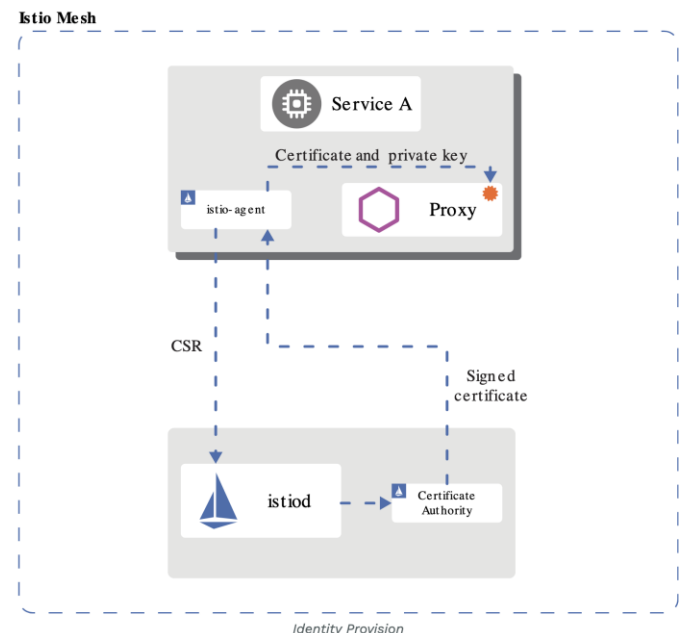
Istio 安全与 SPIFFE:

- Istio 和 SPIFFE 共享相同的身份文件：SVID（SPIFFE 可验证身份证件）。例如：在 Kubernetes 中，X.509 证书的 URI 字段格式为 `spiffe://<domain>/ns/<namespace>/sa/<serviceaccount>`。这使 Istio 服务能够建立和接受与其他 SPIFFE 兼容系统的连接。

SDS

Istio 供应身份是通过 secret discovery service (SDS) 来实现的：

- istiod 提供 gRPC 服务以接受证书签名请求 (CSRs)。
- 当工作负载启动时，Envoy 通过秘密发现服务 (SDS) API 向同容器内的 istio-agent 发送证书和密钥请求。
- 在收到 SDS 请求后，istio-agent 创建私钥和 CSR，然后将 CSR 及其凭据发送到 istiod CA 进行签名。
- istiod CA 验证 CSR 中携带的凭据，成功验证后签署 CSR 以生成证书。
- istio-agent 通过 Envoy SDS API 将私钥和从 Istio CA 收到的证书发送给 Envoy。
- istio-agent 会监工作负载证书的有效期。上述 CSR 过程会周期性地重复，以处理证书和密钥轮换。



认证

基于 Istio 的认证

Istio 通过客户端和服务端 Policy Enforcement Points (PEPs) 建立服务到服务的通信通道。

PEPs 在 Istio 架构中的实现是 Envoy。

Peer authentication:

- 用于服务到服务的认证，以验证进行连接的客户端。

Istio 提供双向 TLS 作为传输认证的全栈解决方案，无需更改服务代码就可以启用它。这个解决方案为：

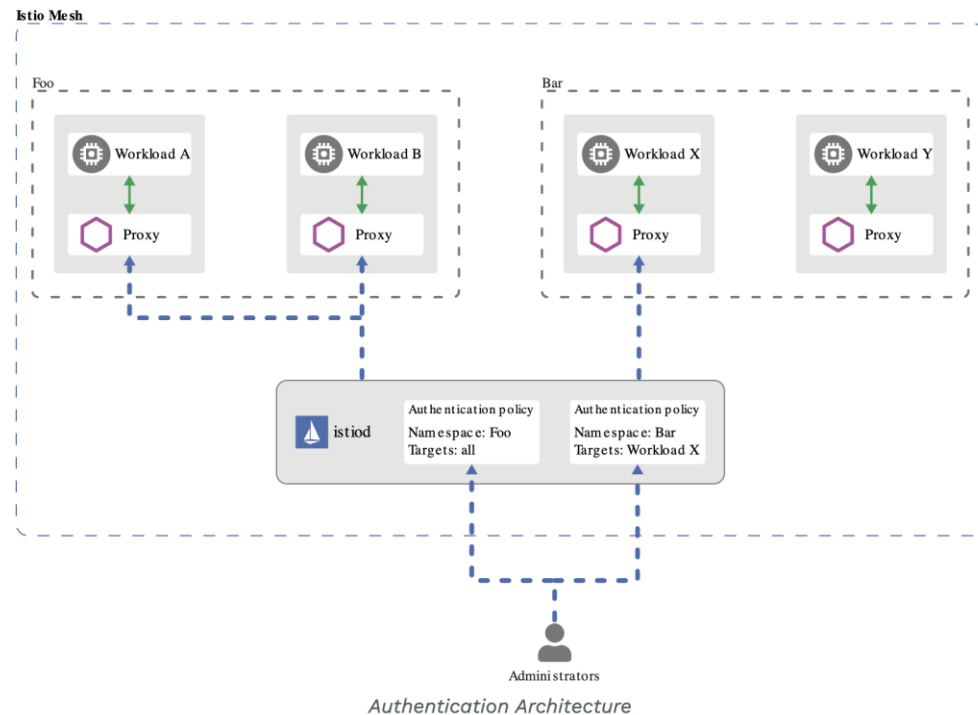
- 为每个服务提供强大的身份，表示其角色，以实现跨群集和云的互操作性。
- 保护服务到服务的通信。
- 提供密钥管理系统，以自动进行密钥和证书的生成，分发和轮换。

Request authentication

- 用于最终用户认证，以验证附加到请求的凭据。
- Istio 使用 JSON Web Token (JWT) 验证启用请求级认证，并使用自定义认证实现或任何 OpenID Connect 的认证实现（例如下面列举的）来简化的开发人员体验。

Istio 认证架构

- 未设置模式的网格范围的 peer 认证策略默认使用 PERMISSIVE 模式。
- 发送请求的客户端服务负责遵循必要的认证机制。
- RequestAuthentication
 - 应用程序负责获取 JWT 凭证并将其附加到请求。
- PeerAuthentication
 - Istio 会自动将两个 PEPs 之间的所有流量升级为双向 TLS。
 - 如果认证策略禁用了双向 TLS 模式，则 Istio 将继续在 PEPs 之间使用纯文本。
 - 要覆盖此行为，destination rules 显式禁用双向 TLS 模式。



双向 TLS 认证

当一个工作负载使用双向 TLS 认证向另一个工作负载发送请求时，该请求的处理方式如下：

- Istio 将出站流量从客户端重新路由到客户端的本地 sidecar Envoy。
- 客户端 Envoy 与服务器端 Envoy 开始双向 TLS 握手。在握手期间，客户端 Envoy 还做了安全命名检查，以验证服务器证书中显示的服务帐户是否被授权运行目标服务。
- 客户端 Envoy 和服务器端 Envoy 建立了一个双向的 TLS 连接，Istio 将流量从客户端 Envoy 转发到服务器端 Envoy。
- 授权后，服务器端 Envoy 通过本地 TCP 连接将流量转发到服务器服务。

宽容模式 (permissive mode)

允许服务同时接受纯文本流量和双向 TLS 流量。

这个功能极大地提升了双向 TLS 的入门体验。

在运维人员希望将服务移植到启用了双向 TLS 的 Istio 上时，许多非 Istio 客户端和非 Istio 服务端通信时会产生问题。

通常情况下，运维人员无法同时为所有客户端安装 Istio sidecar，甚至没有这样做的权限。即使在服务端上安装了 Istio sidecar，运维人员也无法在不中断现有连接的情况下启用双向 TLS。

安全命名

服务器身份 (server identities) 被编码在证书里, 但服务名称 (service names) 通过服务发现或 DNS 被检索。

安全命名信息将服务器身份映射到服务名称。

身份 A 到服务名称 B 的映射表示 “授权 A 运行服务 B”。

控制平面监视 apiserver, 生成安全命名映射, 并将其安全地分发到 PEPs。以下示例说明了为什么安全命名对身份验证至关重要。

认证策略

认证策略是对服务收到的请求生效的；

要在双向 TLS 中指定客户端认证策略，需要在 DestinationRule 中设置 TLSSettings。

```
apiVersion: networking.istio.io/v1beta1
kind: DestinationRule
metadata:
  name: db-mtls
spec:
  host: mydbserver.prod.svc.cluster.local
  trafficPolicy:
    tls:
      mode: MUTUAL
      clientCertificate: /etc/certs/myclientcert.pem
      privateKey: /etc/certs/client_private_key.pem
      caCertificates: /etc/certs/rootcacerts.pem
```

```
apiVersion: networking.istio.io/v1beta1
kind: DestinationRule
metadata:
  name: tls-foo
spec:
  host: "*.foo.com"
  trafficPolicy:
    tls:
      mode: SIMPLE
```

```
apiVersion: networking.istio.io/v1beta1
kind: DestinationRule
metadata:
  name: ratings-istio-mtls
spec:
  host: ratings.prod.svc.cluster.local
  trafficPolicy:
    tls:
      mode: ISTIO_MUTUAL
```

认证策略

相应的需要通过 PeerAuthentication 配置服务端接受何种认证方式

```
apiVersion: "security.istio.io/v1beta1"
kind: "PeerAuthentication"
metadata:
  name: "example-peer-policy"
  namespace: "foo"
spec:
  selector:
    matchLabels:
      app: reviews
  mtls:
    mode: STRICT
```

```
apiVersion: "security.istio.io/v1beta1"
kind: "PeerAuthentication"
metadata:
  name: "example-workload-policy"
  namespace: "foo"
spec:
  selector:
    matchLabels:
      app: example-app
  portLevelMtls:
    80:
      mode: DISABLE
```

RequestAuthentication

Request 认证策略指定验证 JSON Web Token (JWT) 所需的值。这些值包括：

- token 在请求中的位置
- 请求的 issuer
- 公共 JSON Web Key Set (JWKS)

Istio 会根据 request 认证策略中的规则检查提供的令牌（如果已提供），并拒绝令牌无效的请求。

当请求不带有令牌时，默认情况下将接受它们。

RequestAuthentication

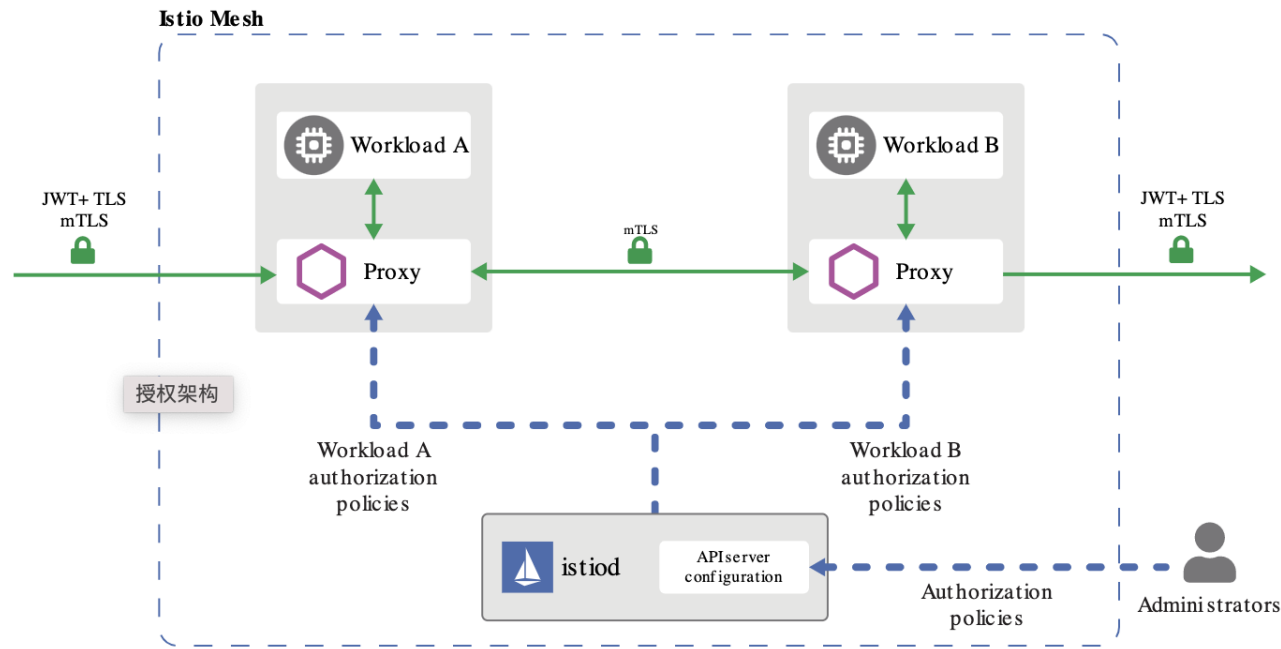
```
apiVersion: security.istio.io/v1beta1
kind: RequestAuthentication
metadata:
  name: "default"
  namespace: istio-system
spec:
  selector:
    matchLabels:
      istio: ingressgateway
  jwtRules:
    - issuer: "testing@secure.istio.io"
      jwksUri: "https://raw.githubusercontent.com/istio/istio/release-1.12/security/tools/jwt/samples/jwks.json"
```

鉴权

授权架构

每个 Envoy 代理都运行一个授权引擎，该引擎在运行时授权请求。

- 当请求到达代理时，授权引擎根据当前授权策略评估请求上下文，并返回授权结果 ALLOW 或 DENY。
- 授权策略支持 ALLOW 和 DENY 动作，拒绝策略优先于允许策略。
- 如果将任何允许策略应用于工作负载，则默认情况下，不符合该策略的访问都将被禁止。



授权策略 (AuthorizationPolicy)

selector 字段指定策略的目标

action 字段指定允许还是拒绝请求

rules 指定何时触发动作

- rules 下的 from 字段指定请求的来源
- rules 下的 to 字段指定请求的操作
- rules 下的 when 字段指定应用规则所需的条件

```
apiVersion: security.istio.io/v1beta1
kind: AuthorizationPolicy
metadata:
  name: httpbin-deny
  namespace: foo
spec:
  selector:
    matchLabels:
      app: httpbin
      version: v1
  action: DENY
  rules:
    - from:
      - source:
        notNamespaces: ["foo"]
```

若非来自 foo, 则拒绝

```
apiVersion: security.istio.io/v1beta1
kind: AuthorizationPolicy
metadata:
  name: httpbin
  namespace: foo
spec:
  selector:
    matchLabels:
      app: httpbin
      version: v1
  action: ALLOW
  rules:
    - from:
      - source:
        principals: ["cluster.local/ns/default/sa/sleep"]
      - source:
        namespaces: ["dev"]
    to:
      - operation:
        methods: ["GET"]
    when:
      - key: request.auth.claims[iss]
        values: ["https://accounts.google.com"]
```

若来自 default/sleep 或 dev, 则允许

OR 关系

策略目标

- 可以通过 metadata/namespace 字段和可选的 selector 字段来指定策略的范围或目标。
- metadata/namespace 告诉该策略适用于哪个命名空间。如果将其值设置为根名称空间，则该策略将应用于网格中的所有名称空间。
- 根命名空间的值是可配置的，默认值为 istio-system。
- 您可以使用 selector 字段来进一步限制策略以应用于特定工作负载。
- 如果未设置，则授权策略将应用于与授权策略相同的命名空间中的所有工作负载。

```
apiVersion:
security.istio.io/v1beta1
kind: AuthorizationPolicy
metadata:
  name: allow-read
  namespace: default
spec:
  selector:
    matchLabels:
      app: products
  action: ALLOW
  rules:
    - to:
        operation:
          methods: ["GET", "HEAD"]
```

值匹配

授权策略中的大多数字段都支持以下所有匹配模式：

- 完全匹配：即完整的字符串匹配。
- 前缀匹配： "*" 结尾的字符串。例如，
"test.abc.*" 匹配 "test.abc.com"、
"test.abc.com.cn"、"test.abc.org" 等等。
- 后缀匹配： "*" 开头的字符串。例如，
"*.abc.com" 匹配 "eng.abc.com"、
"test.eng.abc.com" 等等。
- 存在匹配： * 用于指定非空的任意内容。您可以使用格式 `fieldname: ["*"]` 指定必须存在的字段。这意味着该字段可以匹配任意内容，但是不能为空。

```
apiVersion:
security.istio.io/v1beta1
kind: AuthorizationPolicy
metadata:
  name: tester
  namespace: default
spec:
  selector:
    matchLabels:
      app: products
  action: ALLOW
  rules:
    - to:
        - operation:
            paths: ["/test/*", "*/info"]
```

全部允许和默认全部拒绝授权策略

允许完全访问 default 命名空间中的所有工作负载

```
apiVersion: security.istio.io/v1beta1
kind: AuthorizationPolicy
metadata:
  name: allow-all
  namespace: default
spec:
  action: ALLOW
  rules:
  - {}
```

不允许任何对 admin 命名空间工作负载的访问

```
apiVersion: security.istio.io/v1beta1
kind: AuthorizationPolicy
metadata:
  name: deny-all
  namespace: admin
spec:
  {}
```

自定义条件

```
apiVersion: security.istio.io/v1beta1
kind: AuthorizationPolicy
metadata:
  name: httpbin
  namespace: foo
spec:
  selector:
    matchLabels:
      app: httpbin
      version: v1
  action: ALLOW
```

```
rules:
- from:
  - source:
      principals: ["cluster.local/ns/default/sa/sleep"]
  to:
  - operation:
      methods: ["GET"]
  when:
  - key: request.headers[version]
    values: ["v1", "v2"]
```

认证与未认证身份

如果要使工作负载可公开访问，
则需要将 source 部分留空

```
apiVersion: security.istio.io/v1beta1
kind: AuthorizationPolicy
metadata:
  name: httpbin
  namespace: foo
spec:
  selector:
    matchLabels:
      app: httpbin
      version: v1
  action: ALLOW
  rules:
  - to:
    - operation:
        methods: ["GET", "POST"]
```

将 principal 设置为 “*” 代表仅允许经过认证的用户

```
apiVersion: security.istio.io/v1beta1
kind: AuthorizationPolicy
metadata:
  name: httpbin
spec:
  selector:
    matchLabels:
      app: httpbin
      version: v1
  action: ALLOW
  rules:
  - from:
    - source:
        principals: ["*"]
    to:
    - operation:
        methods: ["GET", "POST"]
```

在普通 TCP 协议上使用 Istio 授权

如果您授权策略中对 TCP 工作负载使用了任何只适用于 HTTP 的字段，Istio 将会忽略它们。

```
apiVersion: "security.istio.io/v1beta1"
kind: AuthorizationPolicy
metadata:
  name: mongodb-policy
  namespace: default
spec:
  selector:
    matchLabels:
      app: mongodb
  action: ALLOW
  rules:
    - from:
        - source:
            principals: ["cluster.local/ns/default/sa/bookinfo-ratings-v2"]
      to:
        - operation:
            ports: ["27017"]
```

THANKS

 极客时间 | 训练营